



مبانی هوش مصنوعی و کاربردها

نیم سال دوم ۱۴۰۵-۱۴۰۴

مهلت: ۳۰ اردیبهشت

استاد:

دکتر الهام هاتفی

دستیاران آموزشی:

محمد صالح مهدی نژاد

مهدی سمسارزاده

صدرا خالقی

راهنمای کامل پروژه PlanForge

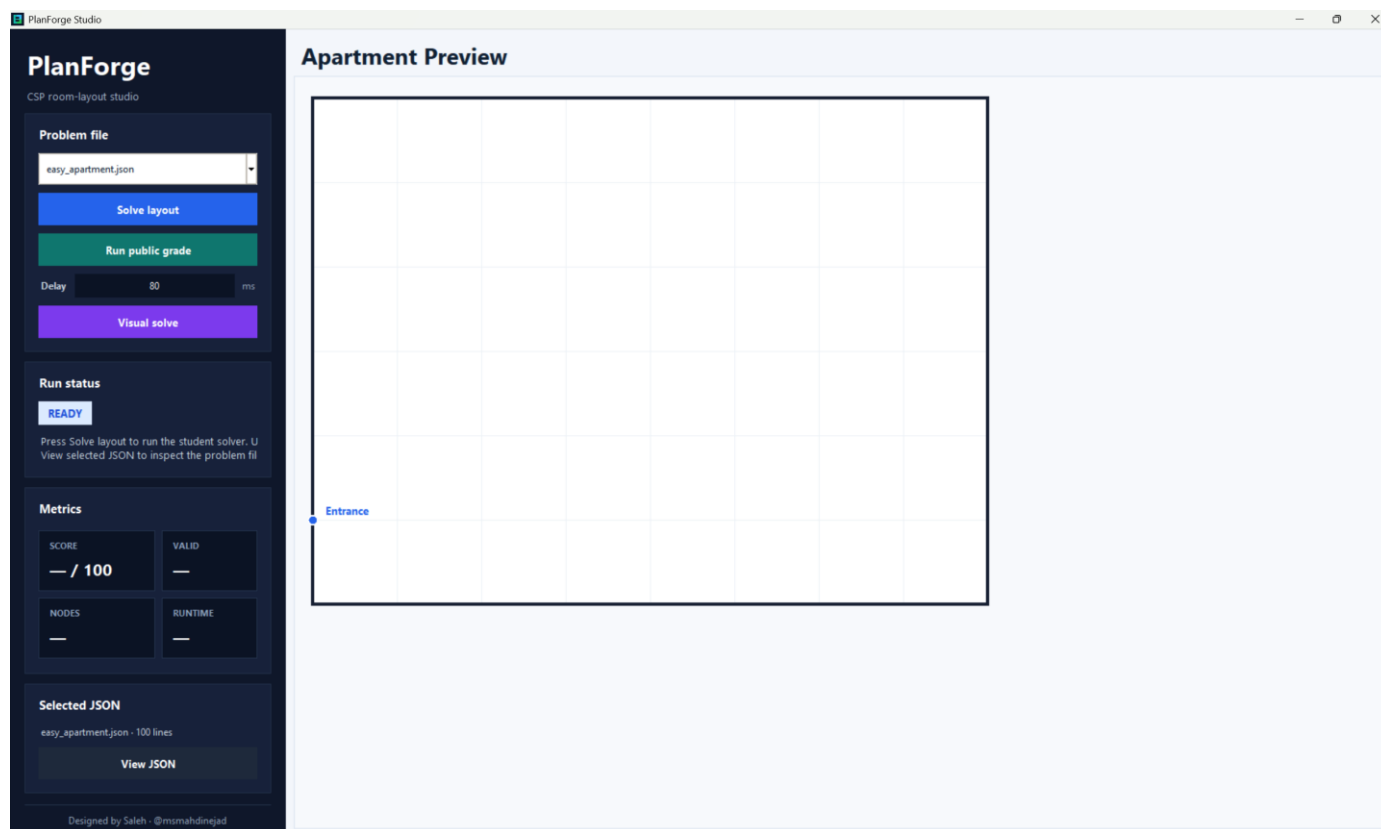
حل‌کننده CSP برای تولید پلان آپارتمان

۱. پروژه دقیقاً چیست؟

در این پروژه، شما هسته یک حل‌کننده CSP را برای چیدن اتاق‌های یک آپارتمان پیاده‌سازی می‌کنید. برنامه از قبل یک رابط گرافیکی دارد و می‌تواند نقشه را نمایش دهد، اما بخش تصمیم‌گیری اصلی را شما باید بنویسید. ورودی برنامه شامل ابعاد خانه، لیست اتاق‌ها، دامنه حالت‌های ممکن برای هر اتاق، و تعدادی قید است. خروجی شما باید مشخص کند هر اتاق در کجای پلان قرار بگیرد.

هر اتاق، مثل پذیرایی، آشپزخانه، اتاق خواب، سرویس یا راهرو، یک متغیر CSP است. دامنه هر متغیر هم مجموعه‌ای از مستطیل‌های ممکن داخل پلان خانه است. برای مثال، اتاق خواب ممکن است بتواند در چند جای مختلف و با چند اندازه مختلف قرار بگیرد. حل‌کننده شما باید از بین این حالت‌ها یک چیدمان معتبر پیدا کند.

قیود سخت پروژه شامل این موارد است: اتاق‌ها نباید روی هم بیفتند، همه اتاق‌ها باید داخل مرز خانه باشند، مساحت هر اتاق باید در بازه مجاز باشد، بعضی اتاق‌ها باید کنار هم باشند، بعضی اتاق‌ها نباید کنار هم باشند، بعضی اتاق‌ها باید به هم نزدیک یا از هم دور باشند، و بعضی فضاها باید به مرز بیرونی دسترسی داشته باشند تا نورگیری داشته باشند.



۲. شروع سریع؛ اگر فقط می‌خواهید پروژه را اجرا کنید...

بعد از اینکه فایل پروژه را از حالت فشرده خارج کردید، وارد پوشه اصلی پروژه شوید. اگر روی ویندوز هستید، ساده‌ترین راه این است که روی فایل **run_windows.bat** دوبار کلیک کنید. اگر می‌خواهید از ترمینال استفاده کنید، دستور زیر را بزنید :

```
python run_app.py
```

برای اجرای public grader، یعنی بررسی اولیه پروژه از سمت خودتان، یکی از این دو روش را استفاده کنید :

```
python run_public_grade.py
```

یا روی ویندوز :

```
run_public_grade.bat
```

اگر خطای مربوط به نسخه Python گرفتید، ابتدا نسخه Python خود را بررسی کنید :

```
python --version
```

پیشنهاد می‌شود از Python ۳/۱۰ یا جدیدتر استفاده کنید. این پروژه برای اجرای معمولی به کتابخانه اضافه نیاز ندارد، چون رابط گرافیکی با tkinter پیاده‌سازی شده است که همراه Python نصب می‌شود. اگر tkinter روی سیستم شما نصب نبود، باید نصب Python را کامل‌تر انجام دهید یا از نسخه رسمی Python استفاده کنید.

۳. وقتی برنامه باز شد، چه کار کنم؟

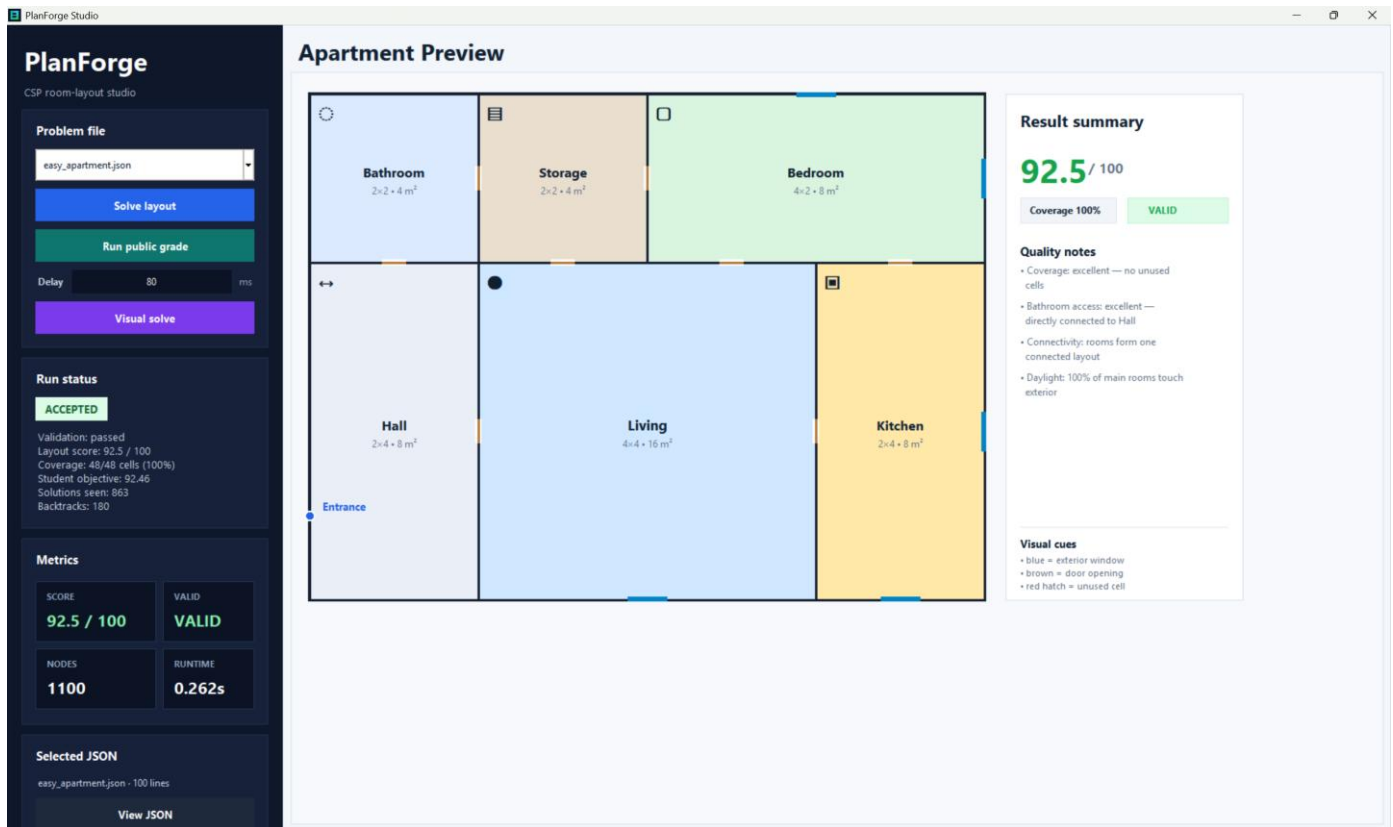
• از بخش Problem file یکی از فایل‌های نمونه، مثل easy_apartment.json یا medium_apartment.json را انتخاب کنید.

• دکمه Solve layout را بزنید. این دکمه کد شما را از پوشه student اجرا می‌کند.

• اگر هنوز TODOها را کامل نکرده باشید، برنامه خطای NotImplementedError نشان می‌دهد. این طبیعی است.

• اگر حل‌کننده شما جواب پیدا کند، پلان در سمت راست نمایش داده می‌شود و در پنل نتیجه می‌توانید اعتبار جواب، score، coverage، تعداد نودها، تعداد backtrackها و زمان اجرا را ببینید.

• دکمه Run public grade یک ارزیابی اولیه انجام می‌دهد. این نمره، نمره نهایی درس نیست، اما برای فهمیدن وضعیت پروژه بسیار مفید است.



۴. حالت Visual Solve

برنامه علاوه بر اجرای عادی، یک حالت نمایشی به نام Visual solve دارد. در اجرای عادی با دکمه Solve layout فقط نتیجه نهایی نمایش داده می‌شود، اما در حالت Visual Solve می‌توانید روند جست‌وجوی الگوریتم خود را به صورت مرحله‌به‌مرحله ببینید.

برای استفاده از این حالت، ابتدا از بخش Problem file یک فایل JSON انتخاب کنید. سپس مقدار Delay را وارد کنید. این مقدار بر حسب میلی‌ثانیه است و مشخص می‌کند بین نمایش هر دو مرحله چقدر فاصله باشد. پس از آن، دکمه Visual solve را بزنید.

در این حالت، برنامه دو چیز را نمایش می‌دهد: اول، تغییرات مرحله‌ای روی پلان آپارتمان؛ دوم، درخت جست‌وجوی DFS در پایین صفحه. این درخت نشان می‌دهد الگوریتم شما چه شاخه‌هایی را امتحان کرده، از کدام مسیرها برگشته، کدام مسیرها prune شده‌اند، و در چه نقطه‌ای به جواب رسیده است. رنگ‌ها و علامت‌های پایین گراف، معنی حالت‌های مختلف نودها را نشان می‌دهند.

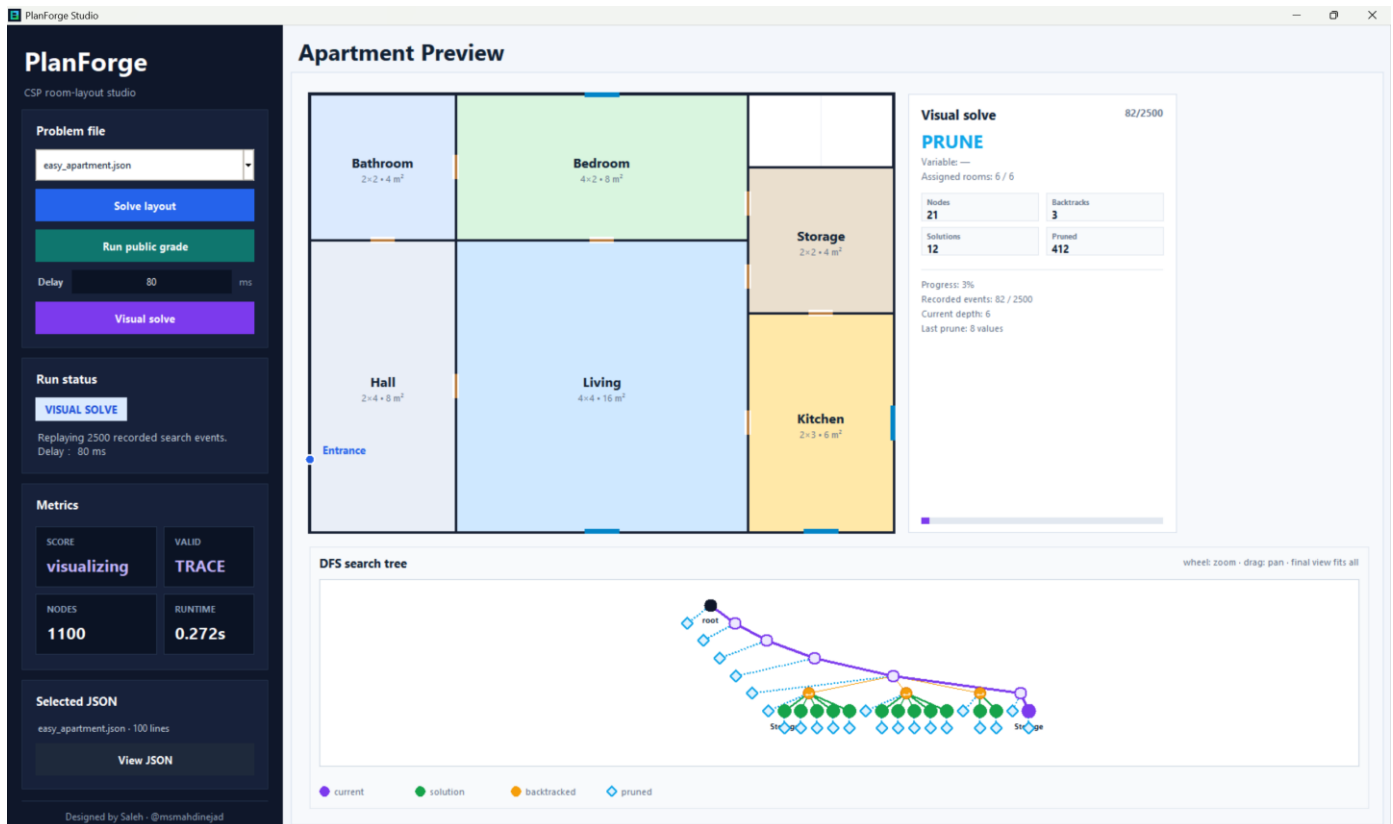
برای اینکه نمایش Visual Solve دقیق‌تر باشد، بهتر است در پیاده‌سازی backtrack از hook‌های نمایشی ctx استفاده کنید:

ctx.on_select_variable(variable, assignment)

ctx.on_assign(variable, value, assignment)

ctx.on_unassign(variable, assignment)

این hook ها جایگزین منطق حل نیستند و فقط برای نمایش بهتر روند اجرا استفاده می‌شوند. اگر آن‌ها را صدا نزنید، ممکن است solver شما همچنان اجرا شود، اما نمایش Visual Solve کامل و قابل فهم نباشد. در گراف DFS می‌توانید با چرخ موس zoom کنید و با drag کردن، گراف را جابه‌جا کنید. این قابلیت بیشتر برای debug کردن و فهمیدن رفتار backtracking، MRV، LCV و pruning استفاده می‌شود.



۴. ساختار پوشه‌ها

ساختار اصلی پروژه به شکل زیر است. **شما فقط اجازه دارید فایل‌های داخل پوشه *student* را تغییر دهید.** فایل‌های داخل planforge برای اجرای برنامه، ساخت مسئله، نمایش گرافیکی و اعتبارسنجی هستند و نباید تغییر داده شوند.

```
student_release/  
  run_app.py  
  run_public_grade.py  
  run_windows.bat  
  run_public_grade.bat  
  
student/  
  solver.py
```

consistency.py
heuristics.py
inference.py
scoring.py

planforge/
core/
models.py
geometry.py
constraints.py
domain_builder.py
loader.py
validator.py
engine.py
quality.py
visualizer.py

examples/
easy_apartment.json
medium_apartment.json
hard_apartment.json
unsat_apartment.json

grader/
public_grader.py
_public_grader...

مسیر	توضیح
student/solver.py	بخش اصلی حل‌کننده. شما باید solve، backtrack و is_complete را کامل کنید.
student/consistency.py	بررسی سازگاری قیود سخت در assignment‌های جزئی.
student/heuristics.py	پیاده‌سازی MRV برای انتخاب متغیر و LCV برای ترتیب مقدارها. این بخش اجباری است.
student/inference.py	پیاده‌سازی forward checking و AC-3. این بخش امتیازی است، ولی برای solver قوی خیلی مهم است.
student/scoring.py	امتیازدهی به پلان‌های معتبر. این بخش امتیازی است و برای انتخاب جواب بهتر استفاده می‌شود.
planforge/core/geometry.py	توابع آماده هندسی. برای حل پروژه لازم نیست هندسه را از صفر بنویسید.
planforge/core/engine.py	اجرای کنترل‌شده solver و ثبت آمار. این فایل را تغییر ندهید.
planforge/examples/	فایل‌های JSON نمونه. این فایل‌ها مسئله‌های آماده برای تست هستند.

۵. دقیقاً چه فایل‌هایی را باید کامل کنید؟

تمام کدی که باید بنویسید داخل پوشه student قرار دارد. در هر فایل، چند تابع با `raise NotImplementedError` مشخص شده است. شما باید همین تابع‌ها را کامل کنید و امضای تابع‌ها را تغییر ندهید. تغییر نام تابع، حذف ورودی‌ها یا تغییر نوع خروجی باعث می‌شود grader نتواند کد شما را اجرا کند.

۵.۱. فایل solver.py

این فایل قلب پروژه است. شما باید الگوریتم backtracking را خودتان پیاده‌سازی کنید. framework فقط یک `ctx` به شما می‌دهد تا آمار اجرا را ثبت کند و بهترین جواب را نگه دارد. شما نباید آمارها را دستی بسازید، اما باید در جای درست متدهای `ctx` را صدا بزنید.

```
def solve(problem, ctx) -> Assignment | None:
    ...

def backtrack(problem, assignment, domains, ctx) -> None:
    ...

def is_complete(problem, assignment) -> bool:
    ...
```

تابع `solve` نقطه شروع حل است. معمولاً در این تابع باید از `ctx.copy_domains()` یک کپی از دامنه‌ها بگیرید، در صورت پیاده‌سازی AC-3 آن را اجرا کنید، یک `assignment` خالی بسازید، `backtrack` را صدا بزنید، و در پایان `ctx.best_assignment` را برگردانید.

```
def solve(problem, ctx):
    domains = ctx.copy_domains()
    # را پیاده کرده‌اید، می‌توانید اینجا اجرا کنید AC-3 اگر
    # domains = ac3(problem, domains)
    # if domains is None:
    #     return None
    backtrack(problem, {}, domains, ctx)
    return ctx.best_assignment
```

تابع `backtrack` باید recursive باشد. هر بار که وارد یک `node` جدید از جست‌وجو می‌شوید، `ctx.on_node()` را صدا بزنید. اگر `assignment` کامل شد، `ctx.on_solution(assignment)` را صدا بزنید. این متد خودش جواب را `validate` می‌کند، `score_assignment` را صدا می‌زند، و اگر جواب بهتر از جواب‌های قبلی بود، آن را ذخیره می‌کند.

نکته مهم درباره `ctx`
اگر جواب کامل پیدا کردید، فقط `return` کردن `assignment` کافی نیست. باید `ctx.on_solution(assignment)` را هم صدا بزنید. اگر این کار را نکنید، framework نمی‌تواند `best_score`، `solutions_seen` و `best_assignment` را درست ثبت کند.

۵.۲. فایل consistency.py

در این فایل باید تابع `is_consistent` را کامل کنید. این تابع بررسی می‌کند که اگر یک مقدار جدید برای یک اتاق انتخاب شود، آیا `assignment` فعلی هنوز با قیود سخت سازگار است یا نه.

```
def is_consistent(problem, assignment, variable, value) -> bool:
    ...
```

این تابع باید حداقل موارد زیر را بررسی کند :

- مستطیل `value` داخل مرز خانه باشد.
 - مساحت اتاق در بازه `min_area` و `max_area` باشد.
 - `value` با هیچ اتاقی که قبلاً `assign` شده است `overlap` نداشته باشد.
 - هر قید `unary` یا `binary` که در حال حاضر قابل بررسی است، نقض نشود.
- برای بررسی قیود `explicit` می‌توانید از تابع آماده `constraint_is_satisfied` استفاده کنید. این تابع اگر هنوز همه متغیرهای یک `constraint assign` نشده باشند، فعلاً `True` برمی‌گرداند. بنابراین برای `assignment` های جزئی مناسب است.

```
from planforge.core.geometry import inside_boundary, overlaps
from planforge.core.constraints import constraint_is_satisfied
```

۵.۳. فایل `heuristics.py`

در این پروژه `MRV` و `LCV` هر دو اجباری هستند. یعنی فقط انتخاب اولین متغیر یا برگرداندن `domain` بدون ترتیب‌دهی کافی نیست.

```
def select_unassigned_variable(problem, assignment, domains) -> str:
    ...

def order_domain_values(problem, variable, assignment, domains) -> list[Rect]:
    ...
```

در `select_unassigned_variable` باید `MRV` را پیاده کنید. `MRV` یعنی متغیری را انتخاب کنید که کمترین تعداد مقدار قابل قبول باقی‌مانده را دارد. برای این کار معمولاً باید برای هر متغیر `assign` نشده، `value` های `domain` آن را بررسی کنید و بشمارید چند `value` با `assignment` فعلی سازگار است.

در `order_domain_values` باید `LCV` را پیاده کنید. `LCV` یعنی `value` هایی را زودتر امتحان کنید که کمترین محدودیت را روی متغیرهای دیگر ایجاد می‌کنند. یک روش ساده این است که برای هر `value` بشمارید اگر آن `value` را انتخاب کنیم، چند `value` از `domain` بقیه متغیرها حذف یا نامعتبر می‌شود. `value` ای که حذف کمتری ایجاد کند، باید زودتر امتحان شود.

۵.۴. فایل `inference.py`

این فایل برای بخش‌های پیشرفته‌تر است. اگر `forward checking` و `AC-3` را پیاده کنید، `solver` شما سریع‌تر و قوی‌تر می‌شود و در بخش امتیازی نمره می‌گیرد.

```
def forward_check(problem, variable, value, assignment, domains):
    ...
```

```
def ac3(problem, domains):
    ...
```

forward_check بعد از assign کردن یک متغیر اجرا می‌شود و domain متغیرهای assign نشده را کوچک می‌کند. مثلاً اگر Living یک مستطیل خاص گرفته است، همه valueهایی از بقیه اتاق‌ها که با Living overlap دارند باید حذف شوند. اگر domain یک متغیر خالی شد، باید None برگردانید.

AC-۳ یک روش preprocessing برای arc consistency است. برای هر جفت متغیر مرتبط، valueهایی را حذف می‌کند که هیچ support معتبری در طرف دیگر ندارند. پیاده‌سازی AC-۳ امتیازی است، اما برای حل مسئله‌های سخت‌تر بسیار مفید است.

۵.۵. فایل scoring.py

تابع score_assignment برای امتیازدهی به پلان‌های معتبر استفاده می‌شود. یک جواب valid لازم نیست حتماً full coverage باشد. اما solver قوی باید بین چند جواب valid، پلان بهتری را انتخاب کند. این کار با score_assignment انجام می‌شود.

```
def score_assignment(problem, assignment) -> float:
    ...
```

وقتی backtracking به یک assignment کامل برسد و شما ctx.on_solution(assignment) را صدا بزنید، framework ابتدا جواب را validate می‌کند. اگر جواب valid باشد، score_assignment را صدا می‌زند. اگر score این جواب از بهترین score قبلی بیشتر باشد، همان جواب به عنوان best_assignment ذخیره می‌شود.

در score_assignment بهتر است به این معیارها توجه کنید: نزدیک بودن پذیرایی به ورودی، دورتر بودن اتاق خواب‌ها از ورودی، اتصال منطقی آشپزخانه و پذیرایی، دسترسی مناسب سرویس از راهرو یا فضای عمومی، نورگیری اتاق‌های اصلی، شکل مناسب اتاق‌ها، و استفاده بهتر از فضا.

۶. ctx چیست و چطور باید از آن استفاده کنم؟

ctx یک شیء آماده است که framework به solve شما می‌دهد. کار ctx این است که اجرای backtracking شما را پایش کند، آمار را ثبت کند، جواب‌های کامل را validate کند، و بهترین جواب را نگه دارد. شما نباید خودتان report بسازید، اما باید eventهای مهم را به ctx اعلام کنید.

توضیح	کجا استفاده شود؟	متد ctx
تعداد nodeهای باز شده را زیاد می‌کند.	ابتدای هر فراخوانی backtrack	ctx.on_node()
تعداد تلاش برای assign کردن را ثبت می‌کند.	قبل از امتحان کردن هر value	ctx.on_assignment_tried()
تعداد بررسی‌های سازگاری را ثبت می‌کند.	قبل از صدا زدن is_consistent	ctx.on_consistency_check()
تعداد valueهای حذف شده از domain را ثبت می‌کند.	بعد از pruning	ctx.on_prune(before, after)

ctx.on_solution(assignment)	وقتی assignment کامل شد	جواب را validate و score می‌کند و اگر بهترین بود ذخیره می‌کند.
ctx.on_backtrack()	وقتی از یک مسیر برمی‌گردید	تعداد backtrackها را ثبت می‌کند.
ctx.should_stop	در ابتدای backtrack یا داخل حلقه	اگر limit اجرا پر شده باشد، باید جست‌وجو را متوقف کنید.
ctx.best_assignment	در پایان solve	بهترین جواب معتبر پیداشده تا آن لحظه.

۷. مدل‌های آماده‌ای که باید بشناسید

برای حل پروژه لازم نیست کلاس‌های جدید زیادی بسازید. مهم‌ترین کلاس‌ها از قبل آماده هستند و از طریق problem به شما داده می‌شوند.

کاربرد	فیلدها یا ویژگی‌های مهم	کلاس یا نوع
نمایش مکان و اندازه یک اتاق در پلان.	x, y, w, h, area, x ² , y ²	Rect
مشخصات مجاز هر اتاق.	,name, min_area, max_area, min_w, max_w, min_h, max_h, needs_exterior	RoomSpec
نمایش یک قید مثل adjacent یا near.	type, variables, params	Constraint
کل مسئله CSP که به توابع شما داده می‌شود.	,width, height, entrance, variables, room_specs, domains, constraints	CSPProblem
دیکشنری‌ای که برای هر اتاق یک Rect نگه می‌دارد.	dict[str, Rect]	Assignment

۸. تابع‌های آماده هندسی؛ لازم نیست هندسه را از صفر بنویسید

یکی از هدف‌های پروژه این است که تمرکز شما روی CSP باشد، نه روی محاسبات هندسی. برای همین تعداد زیادی تابع آماده در planforge.core.geometry قرار دارد. استفاده از این تابع‌ها توصیه می‌شود.

نمونه کاربرد	معنی	تابع
در is_consistent برای رد کردن مستطیل بیرون از خانه.	بررسی می‌کند مستطیل r داخل مرز خانه باشد.	inside_boundary(r, width, height)

overlaps(a, b)	بررسی می‌کند دو مستطیل هم‌پوشانی دارند یا نه.	برای no_overlap بین اتاق جدید و اتاق‌های قبلی.
adjacent(a, b)	بررسی می‌کند دو مستطیل دیوار مشترک دارند یا نه.	برای قید adjacent بین Kitchen و Living.
shared_wall_length(a, b)	طول دیوار مشترک دو اتاق را می‌دهد.	برای score دادن به ارتباط Kitchen و Living.
center(r)	مختصات مرکز مستطیل را می‌دهد.	برای محاسبه فاصله یا معیارهای soft.
manhattan_rect_distance(a, b)	فاصله منتهن بین دو مستطیل را حساب می‌کند.	برای near، far یا دوری Bathroom از Kitchen.
distance_to_point(r, p)	فاصله مرکز یک اتاق تا یک نقطه را می‌دهد.	برای نزدیکی Living یا Hall به entrance.
touches_boundary(r, width, height)	بررسی می‌کند اتاق به مرز بیرونی خانه می‌رسد یا نه.	برای نورگیری Bedroom یا Living.
touches_wall(r, wall, width, height)	بررسی تماس با دیوار north، south، west یا east.	برای قیود خاص‌تر نورگیری.
aspect_ratio(r)	نسبت ضلع بزرگ‌تر به ضلع کوچک‌تر را می‌دهد.	برای جریمه اتاق‌های خیلی کشیده.
used_area(rects)	مجموع مساحت مستطیل‌ها را می‌دهد.	برای score و coverage.
bounding_box(rects)	کوچک‌ترین مستطیلی را می‌دهد که همه اتاق‌ها را دربر می‌گیرد.	برای compactness.
adjacency_graph(assignment)	گراف مجاورت اتاق‌ها را می‌سازد.	برای تحلیل اتصال layout.
is_connected_layout(assignment)	بررسی می‌کند گراف مجاورت متصل است یا نه.	برای valid بودن یا score.

۹. تابع آماده constraint_is_satisfied

در planforge.core.constraints تابع مهمی به نام constraint_is_satisfied وجود دارد. این تابع یک constraint و یک assignment را می‌گیرد و بررسی می‌کند آیا آن قید رعایت شده است یا نه.

```
from planforge.core.constraints import constraint_is_satisfied
```

این تابع از قیدهای زیر پشتیبانی می‌کند :

• touches_boundary : اتاق به مرز بیرونی خانه برسد.

• touches_wall : اتاق به دیوار مشخص شده برسد.

• near_entrance : اتاق به ورودی نزدیک باشد.

• far_from_entrance : اتاق از ورودی فاصله کافی داشته باشد.

• adjacent : دو اتاق کنار هم باشند.

• not_adjacent : دو اتاق کنار هم نباشند.

• near : دو اتاق نزدیک هم باشند.

• far : دو اتاق از هم دور باشند.

• larger_than : یک اتاق از اتاق دیگر بزرگتر باشد.

نکته مهم این است که no_overlap به صورت یک constraint صریح در JSON نمی‌آید، اما همیشه باید رعایت شود. بنابراین خودتان باید در is_consistent بررسی کنید اتاق جدید با اتاق‌های قبلی overlap نداشته باشد.

۱۰. دامنه‌ها از کجا می‌آیند؟

شما لازم نیست domain‌ها را بسازید. فایل domain_builder.py قبل از اجرای solver برای هر اتاق همه Rect‌های ممکن را تولید می‌کند. این Rect‌ها داخل problem.domains قرار دارند.

```
domains = problem.domains
all_living_candidates = domains["Living"]
```

هر value در domain یک Rect است. domain‌ها از قبل بر اساس اندازه خانه، minArea، maxArea، minW، maxW، minH، maxH و بعضی محدودیت‌های اولیه ساخته شده‌اند. با این حال، در is_consistent بهتر است باز هم داخل مرز بودن و درست بودن مساحت را چک کنید تا کد شما robust باشد.

۱۱. فرق hard constraint و soft constraint چیست؟

hard constraint یعنی اگر نقض شود، جواب اصلاً قابل قبول نیست. برای مثال overlap داشتن دو اتاق یا بیرون زدن اتاق از مرز خانه باعث invalid شدن جواب می‌شود. soft constraint یعنی جواب همچنان معتبر است، اما کیفیت آن پایین‌تر است. برای مثال full coverage بودن برای valid بودن اجباری نیست، اما اگر پلان فضای پرت کمتری داشته باشد، امتیاز بهتری می‌گیرد.

اثر در نمره	مثال	نوع قید
اگر نقض شود، جواب invalid است.	اتاق‌ها overlap نداشته باشند.	قید سخت
اگر نقض شود، جواب invalid است.	Living و Kitchen باید adjacent باشند، اگر در JSON آمده باشد.	قید سخت
اگر بهتر رعایت شود، score بالاتر می‌رود.	Living نزدیک entrance باشد.	قید نرم

اگر بهتر رعایت شود، score بالاتر می‌رود.	Bedroom ها private تر و دورتر از entrance باشند.	قید نرم
برای بخش امتیازی مهم است، اما valid بودن پایه را تعیین نمی‌کند.	coverage بیشتر و فضای پرت کمتر باشد.	قید نرم

۱۲. بخش اجباری و بخش امتیازی

نمره اصلی پروژه از بخش اجباری محاسبه می‌شود و از ۱۰۰ است. بخش امتیازی جداگانه است و حداکثر ۳۰ نمره دارد. public grader برای راهنمایی است، اما نمره نهایی با private grader داده می‌شود.

وضعیت	موارد مورد انتظار	بخش
باید کامل شود.	is_complete و solve، backtrack	اجباری
باید کامل شود.	is_consistent و بررسی hard constraints	اجباری
باید کامل شود.	MRV در select_unassigned_variable	اجباری
باید کامل شود.	LCV در order_domain_values	اجباری
باید کامل شود.	استفاده درست از ctx در backtracking	اجباری
باید رعایت شود.	تشخیص درست مسئله بدون جواب	اجباری
اختیاری، اما مهم.	forward checking	امتیازی
اختیاری، برای solver قوی.	AC-۳	امتیازی
اختیاری، برای کیفیت پلان.	score_assignment و انتخاب جواب بهتر	امتیازی
اختیاری.	کارایی بهتر روی hard case ها	امتیازی

۱۳. public grader و private grader

public grader ابزاری است که به شما کمک می‌کند قبل از تحویل بفهمید پروژه در چه وضعیتی است. public grader نمره نهایی نیست و تست‌های آن با private grader فرق دارد. نمره نهایی توسط private grader و تست‌های مخفی محاسبه می‌شود.

برای اجرای public grader بنویسید :

```
python run_public_grade.py
```

```

C:\WINDOWS\system32\cmd. x + v
This public grader is only a self-check. Final grades are computed with a different private grader and hidden cases.

Required: 100.0 / 100.0 (100.0%)
Optional : 16.0 / 30.0 (53.33%)
Total : 116.0 / 130.0 (89.23%)

[PASS] REQUIRED - Required API imports: 10 / 10
all required student functions are importable
[PASS] REQUIRED - MRV/LCV smoke check: 15 / 15
MRV=ok, LCV=ok
[PASS] REQUIRED - Valid layout on easy_apartment.json: 22 / 22
valid=True, quality=81.7/100, nodes=7, solutions=1
[PASS] REQUIRED - Valid layout on medium_apartment.json: 22 / 22
valid=True, quality=78.87/100, nodes=8, solutions=1
[PASS] REQUIRED - Unsat detection: 12 / 12
assignment_found=False, status=no_solution
[PASS] REQUIRED - Backtracking instrumentation: 9 / 9
nodes=8, assignments=188, backtracks=7, checks=188
[PASS] REQUIRED - Project package sanity: 10 / 10
example files present
[PASS] OPTIONAL - Hard-case layout quality: 8 / 12
quality=84.68/100, valid=True, notes=['Coverage: acceptable but improvable (85.4%)', 'Bathroom access: excellent - directly connected to Hall', 'Connectivity: rooms form one connected layout', 'Daylight: 100% of main rooms touch exterior']
[FAIL] OPTIONAL - Optimization / pruning signals: 0.0 / 10
solutions_seen=1, pruned_values=0
[PASS] OPTIONAL - Soft score function: 8 / 8
score_assignment returned numeric value: 0.0

JSON report saved to: C:\Users\SALEH\OneDrive\Desktop\PlanForge_Instructor_Package_v20\instructor_private\reference_basic_solution\public_grade_report.json
Press any key to continue . . .

```

اگر public grader خطا داد، ابتدا متن خطا را بخوانید. رایج‌ترین خطاها این‌ها هستند : تابعی هنوز
 signature تابع را تغییر داده‌اید، solve به جای assignment معتبر None برمی‌گرداند، یا
 ctx.on_solution را صدا نزده‌اید.

۱۴. فایل‌های JSON نمونه را چطور بخوانم؟

هر فایل JSON یک مسئله CSP را تعریف می‌کند. فیلد width و height اندازه خانه را مشخص می‌کنند. فیلد entrance نقطه ورودی خانه را مشخص می‌کند. بخش rooms لیست اتاق‌ها و محدودیت اندازه آن‌هاست. بخش constraints قیود سخت مسئله را مشخص می‌کند.

```

{
  "width": 10,
  "height": 8,
  "entrance": [0, 7],
  "rooms": [
    {"name": "Living", "minArea": 18, "maxArea": 22},
    {"name": "Kitchen", "minArea": 8, "maxArea": 10}
  ],
  "constraints": [
    {"type": "adjacent", "a": "Living", "b": "Kitchen"}
  ]
}

```

برای حل پروژه لازم نیست parser بنویسید. loader.py این فایل‌ها را می‌خواند و یک CSPProblem آماده به تابع solve شما می‌دهد.

۱۵. پیشنهاد مرحله به مرحله برای پیاده‌سازی

پیشنهاد می‌شود پروژه را یک‌باره ننویسید. این ترتیب معمولاً ساده‌تر است :

• مرحله ۱ : is_complete را پیاده کنید.

- مرحله ۲ : `is_consistent` را فقط با `area`، `inside_boundary` و `no_overlap` کامل کنید.
- مرحله ۳ : `constraint_is_satisfied` را هم به `is_consistent` اضافه کنید.
- مرحله ۴ : یک `backtracking` ساده بنویسید که اولین جواب معتبر را پیدا کند.
- مرحله ۵ : `ctx.on_node`، `ctx.on_assignment_tried`، `ctx.on_consistency_check`، `ctx.on_solution` و `ctx.on_backtrack` را در جای درست اضافه کنید.
- مرحله ۶ : `MRV` را در `select_unassigned_variable` کامل کنید.
- مرحله ۷ : `LCV` را در `order_domain_values` کامل کنید.
- مرحله ۸ : `public grader` را اجرا کنید و خطاهای بخش اجباری را رفع کنید.
- مرحله ۹ : اگر بخش امتیازی می‌خواهید، `forward checking` و سپس `AC-۳` را اضافه کنید.
- مرحله ۱۰ : `score_assignment` را بنویسید تا `solver` بین جواب‌های معتبر، پلان بهتر را انتخاب کند.

۱۶. اشتباهات رایج

اشتباه	نتیجه	راه‌حل
تغییر دادن امضای تابع‌ها	<code>grader</code> نمی‌تواند تابع را صدا بزند.	نام تابع و ورودی‌ها را دقیقاً نگه دارید.
صدا نزدن <code>ctx.on_solution</code>	جواب پیدا می‌شود، اما <code>best_assignment</code> و <code>score</code> ثبت نمی‌شود.	هر وقت <code>assignment</code> کامل شد، <code>ctx.on_solution</code> را صدا بزنید.
بررسی نکردن <code>overlap</code>	جواب <code>invalid</code> می‌شود.	در <code>is_consistent</code> با <code>overlaps</code> همه اتاق‌های قبلی را چک کنید.
استفاده نکردن از <code>MRV</code> و <code>LCV</code>	نمره اجباری کامل نمی‌شود.	هر دو <code>heuristic</code> در نسخه فعلی اجباری هستند.
حذف نکردن مقدار از <code>assignment</code> بعد از <code>recursion</code>	مسیرهای بعدی خراب می‌شوند.	حتماً <code>recursive call</code> بعد از <code>assignment.pop(var)</code> بزنید.
تغییر فایل‌های <code>planforge</code>	<code>private grader</code> تغییرات شما را در نظر نمی‌گیرد.	فقط پوشه <code>student</code> را تغییر دهید.
فرض کردن اینکه <code>full coverage</code> اجباری است	ممکن است بی‌دلیل جواب‌های <code>valid</code> را رد کنید.	<code>full coverage</code> یک هدف نرم و امتیازی است، نه شرط <code>valid</code> بودن پایه.

۱۷. تحویل پروژه

برای تحویل پروژه، لازم است موارد زیر را با دقت آماده و در کوئرا بارگذاری کنید.

ابتدا پوشه‌ی student خود را به صورت یک فایل ZIP ارسال کنید. نام فایل ZIP باید دقیقاً به فرمت زیر باشد:

PlanForge.zip_شماره دانشجویی

برای مثال:

402123456_PlanForge.zip

دقت کنید که فقط پوشه‌ی student باید ZIP شود، نه کل پروژه. در صورتی که فایل ZIP شامل ساختار اشتباه، فایل‌های اضافه، یا کل پروژه باشد، ممکن است فرآیند نمره‌دهی با مشکل مواجه شود و بخشی از نمره را از دست بدهید.

علاوه بر فایل ZIP، لازم است تمام فایل‌های داخل پوشه‌ی student را به صورت جداگانه نیز در کوئرا بارگذاری کنید. این فایل‌ها برای بررسی دقیق‌تر و اجرای فرآیند کشف تقلب استفاده می‌شوند. بنابراین، هم فایل ZIP پوشه‌ی student و هم تک‌تک فایل‌های موجود در آن باید ارسال شوند.

همچنین باید یک فایل گزارش جداگانه به صورت PDF تهیه و در کوئرا بارگذاری کنید. در این گزارش باید توضیح دهید که چه بخش‌هایی را پیاده‌سازی کرده‌اید، منطق کد شما چیست، از چه روش‌هایی برای حل مسئله استفاده کرده‌اید، و چرا این روش‌ها را انتخاب کرده‌اید. گزارش باید شامل توضیح کد، نمایش خروجی‌ها، تحلیل نتایج، و در صورت پیاده‌سازی بخش‌های امتیازی، توضیح کامل آن‌ها باشد.

نام فایل گزارش نیز باید پیش از ارسال، به شماره‌ی دانشجویی شما تغییر داده شود. برای مثال:

402123456.pdf

در صورتی که نام فایل گزارش مطابق فرمت خواسته شده نباشد، ممکن است بخشی از نمره‌ی شما کسر شود. فایل گزارش نهایی را در کوئرا و در تمرینی با نام «گزارش» بارگذاری کنید.

برای ارائه‌ی پروژه، باید روی تمام بخش‌هایی که پیاده‌سازی کرده‌اید تسلط کامل داشته باشید. لازم است بتوانید توضیح دهید هر تابع چه کاری انجام می‌دهد، چرا از روش‌های موردنظر خود استفاده کرده‌اید، الگوریتم شما چگونه کار می‌کند، و خروجی‌های تولیدشده چه معنایی دارند. همچنین این توضیحات باید در گزارش شما نیز آورده شده باشند. عدم تسلط بر کد یا ناتوانی در توضیح منطق پیاده‌سازی می‌تواند در نمره‌ی نهایی تأثیر منفی داشته باشد.

موفق باشید!